

Representación Computacional de Números Reales

Diego Feroldi
feroldi@fceia.unr.edu.ar

Arquitectura del Computador*
Departamento de Ciencias de la Computación
FCEIA-UNR



* Actualizado 15 de septiembre de 2026 (D. Feroldi, feroldi@fceia.unr.edu.ar)

Índice

1. Introducción	1
2. Representación de números reales con punto fijo	1
2.1. Sistemas de numeración posicionales	1
2.2. Rango y precisión en la representación en punto fijo	2
2.3. Ventajas y desventajas de la representación en punto fijo	3
3. Representación de números reales con punto flotante	4
3.1. Notación científica normalizada	4
3.2. Características de los números en notación científica normalizada	6
3.3. Rango y precisión en la representación en punto flotante	8
3.4. Distribución no uniforme	9
3.5. Épsilon de máquina	10
3.6. Ventajas y limitaciones de la representación en punto flotante	11
4. Estándar IEEE 754 para números en punto flotante	12
4.1. Representación de números normalizados en IEEE 754	13
4.2. Formatos IEEE 754	14
4.3. Exponente sesgado	14
4.4. Significando	16
4.5. Conversión de decimal a IEEE 754	17
4.6. Conversión de IEEE 754 a decimal	18
4.7. Valores especiales en punto flotante (IEEE 754)	19
4.7.1. Números denormalizados	20
4.7.2. Ceros	20
4.7.3. Infinitos	21
4.7.4. Valor NaN	22
4.8. Rango y precisión en la representación IEEE 754	23
5. Operaciones con números en punto flotante	25
5.1. Suma/resta	25
5.2. Multiplicación	26
5.3. División	28
6. Error en la representación de números reales	29
6.1. Representación finita en IEEE 754	29
6.2. Redondeo de un número real	29
6.3. Redondeo en IEEE 754	30
6.4. Error de representación	30

6.5. Precauciones y consecuencias al operar con números en punto flotante	32
A. Apéndices	35
A.1. Formatos y características de punto flotante (IEEE 754) . . .	35
A.2. Cálculo del épsilon de máquina	35
A.3. Ejemplo en C: parámetros en punto flotante	37
A.4. Manipulación de la representación binaria de números en pun- to flotante	39

1. Introducción

La representación de números reales en binario plantea desafíos particulares debido a la necesidad de expresar tanto la parte entera como la parte fraccionaria en una base que sólo utiliza dos dígitos: 0 y 1. A diferencia del sistema decimal, donde las fracciones comunes (como $1/10$ o $1/2$) tienen una representación sencilla, en binario muchas fracciones decimales no pueden representarse de forma exacta utilizando un número finito de bits. Por ejemplo, el número decimal 0.1 tiene una expansión binaria infinita, lo que introduce errores de redondeo en su representación digital.

Para abordar esta problemática, se emplean dos estrategias principales:

- **Punto fijo:** Se reserva una cantidad fija de bits para la parte entera y otra para la parte fraccionaria. La posición del punto binario se asume fija y no se almacena de forma explícita.
- **Punto flotante:** Se adopta una representación similar a la notación científica, donde un número se expresa como el producto de un significando y una potencia de 2. En este caso, la posición del punto binario se controla mediante un exponente almacenado explícitamente. El formato de punto flotante binario más utilizado es el definido por el estándar IEEE 754 que veremos en la Sección 4.

2. Representación de números reales con punto fijo

Este método se basa en el sistema de numeración posicional presentado en el apunte anterior, donde se consideró el caso particular de números enteros. En la representación con punto fijo, se supone que el punto (o coma) se encuentra siempre en una posición predeterminada. Aunque este no se almacena explícitamente, su ubicación se asume implícitamente. De este modo, los dígitos a la izquierda de dicha posición representan la parte entera (potencias no negativas de la base), mientras que los dígitos a la derecha representan la parte fraccionaria (potencias negativas).

2.1. Sistemas de numeración posicionales

La definición general de un sistema de numeración posicional en base $\beta \geq 2$ es

$$(-1)^s(a_n a_{n-1} \dots a_1 a_0 . a_{-1} a_{-2} \dots)_\beta,$$

la cual representa el número real

$$(-1)^s (a_n \beta^n + a_{n-1} \beta^{n-1} + \cdots + a_1 \beta^1 + a_0 \beta^0 + a_{-1} \beta^{-1} + a_{-2} \beta^{-2} + \cdots),$$

donde s determina el signo del número ($s = 0$ si es positivo y $s = 1$ si es negativo), y los coeficientes a_i son los dígitos en base β , cumpliendo $0 \leq a_i \leq \beta - 1$. Los coeficientes a_i con $i \geq 0$ se consideran como los dígitos de la parte entera, en tanto que los a_i con $i < 0$, son los dígitos de la parte fraccionaria.

De este modo, el sistema decimal corresponde al caso particular $\beta = 10$. De manera análoga, pueden definirse sistemas posicionales en cualquier otra base; por ejemplo, el sistema binario, que corresponde a $\beta = 2$.

2.2. Rango y precisión en la representación en punto fijo

En la representación en punto fijo, la posición del punto (o coma) se mantiene constante. Esto implica que la cantidad de bits (o dígitos) asignados a la parte entera y a la parte fraccionaria está predeterminada. Como consecuencia, el rango de valores representables y la precisión quedan completamente definidos por esta elección.

El **rango** está determinado por la cantidad de bits asignados a la parte entera. Cuantos más bits se destinan a esta parte, mayor es el valor máximo que puede representarse. Sin embargo, esto reduce la cantidad de bits disponibles para la parte fraccionaria.

Por otro lado, la **precisión** depende de la cantidad de bits asignados a la parte fraccionaria. Esta determina la mínima diferencia entre dos valores representables consecutivos, también conocida como resolución. A mayor cantidad de bits fraccionarios, mayor precisión, pero a costa de un menor rango.

Existe, por lo tanto, un compromiso entre rango y precisión: aumentar uno implica necesariamente reducir el otro. En particular, la precisión es uniforme en todo el rango de representación, ya que la separación entre valores consecutivos es constante. Esto contrasta con la representación en punto flotante, que se estudiará más adelante, donde la precisión relativa varía según el orden de magnitud del número representado.

Observación

En este tipo de representación en punto fijo, el punto binario no se almacena explícitamente, sino que se asume en una posición fija predeterminada.

Sin embargo, en los ejemplos lo incluimos para facilitar la lectura e interpretación del número.

Ejemplo

Consideremos un sistema de representación en punto fijo de 8 bits con convención magnitud–signo: 1 bit para el signo, 4 bits para la parte entera y 3 bits para la parte fraccionaria.

Por ejemplo:

$$\begin{aligned}[1 \mid 1011 \mid 011] &= (-1)^1 \times (1011.011)_2 \\ &= (-1)^1 \times (1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ &\quad + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3}) \\ &= (-11.375)_{10}\end{aligned}$$

- **Mayor número representable:**

$$(01111.111)_2 = 2^4 - 2^{-3} = (15.875)_{10}$$

- **Menor número representable:**

$$(11111.111)_2 = -(2^4 - 2^{-3}) = (-15.875)_{10}$$

- **Precisión (resolución):**

$$(00000.001)_2 = 2^{-3} = (0.125)_{10}$$

2.3. Ventajas y desventajas de la representación en punto fijo

Ventajas: La aritmética de punto fijo es relativamente simple.

Desventajas: El rango y la precisión de representación son muy limitados.

Observaciones

- La representación en punto fijo presenta limitaciones significativas tanto en el rango de valores que puede expresar como en la precisión que

ofrece. Por este motivo, resulta necesario emplear sistemas de representación más flexibles, como el punto flotante, que permitan superar estas restricciones.

- El uso de la representación en punto fijo resulta adecuado en sistemas embebidos, microcontroladores sin unidad de coma flotante (FPU) o hardware especializado (como FPGAs, *Field-Programmable Gate Array*), donde se requiere procesar números reales con recursos limitados y con un control preciso del costo computacional.

3. Representación de números reales con punto flotante

“Punto flotante, o cómo aumentar el rango y la precisión al mismo tiempo... ¡y no es magia!”

Como hemos visto en la sección anterior, la representación en punto fijo presenta importantes limitaciones en cuanto al rango y la precisión. A continuación, veremos cómo estas limitaciones pueden superarse utilizando la notación científica, que constituye la base del sistema de representación de números reales en punto flotante.

3.1. Notación científica normalizada

Para la representación de números reales sobre un amplio rango de valores se emplea la notación científica.

Ejemplo

El número 976000000000000 se puede representar como 9.76×10^{14} mientras que el número 0.000000000000000976 se puede representar como 9.76×10^{-16} .

En esta notación la coma o punto decimal se mueve dinámicamente a una posición conveniente y se utiliza el exponente en base 10 para registrar la posición del punto decimal.

Sin embargo, observemos que este tipo de notación tiene cierta ambigüedad dado que un mismo número puede ser representado de diferentes maneras. Por lo tanto, es necesario definir una normalización.

Definición

Todo número real no nulo puede escribirse de manera única en **notación científica normalizada** en base $\beta \geq 2$ como

$$N = \pm(a_0.a_{-1}a_{-2}a_{-3}\dots)_\beta \times \beta^e = (-1)^s \times m \times \beta^e,$$

donde

$$m = a_0 + \sum_{i=1}^{\infty} a_{-i}\beta^{-i}$$

se denomina **significando**, e es el **exponente**, y s indica el **signo** (con $s = 0$ si N es positivo y $s = 1$ si es negativo). Los dígitos a_i satisfacen $0 \leq a_i \leq \beta - 1$.

La condición de normalización es $1 \leq a_0 \leq \beta - 1$, lo que garantiza la unicidad de la representación.

Observación

En la literatura es habitual referirse al significando como “mantisa”. Sin embargo, este uso no es estrictamente correcto: el término “mantisa” proviene de la teoría de logaritmos (donde denota la parte fraccionaria del logaritmo), mientras que en aritmética de punto flotante, conforme a la norma IEEE 754, el término apropiado es *significando* o *significante*.

Ejemplo

La forma normalizada del número 976000000000000 es 9.76×10^{14} , ya que el significando pertenece al intervalo $[1, 10)$.

En el caso particular del sistema binario, donde $\beta = 2$, el único valor posible para el dígito a la izquierda del punto es $a_0 = 1$. Así, el significando resulta $m = (1.a_{-1}a_{-2}\dots a_{-t})_2$. Esto se puede expresar como:

$$m = 1 + a_{-1} \times 2^{-1} + a_{-2} \times 2^{-2} + \dots + a_{-t} \times 2^{-t}$$

Por lo tanto, el número representado corresponde al número decimal

$$N = (-1)^s \times m \times 2^e$$

Ejemplo

Dado el número en formato decimal $N = (3.375)_{10}$, se puede representar en binario como $(11.011)_2$, dado que $3_{10} = (11)_2$ y $(0.375)_{10} = (0.011)_2$. Este último resultado se obtiene mediante multiplicaciones sucesivas de la parte fraccionaria por 2. En cada paso, la parte entera del resultado corresponde a un bit de la representación binaria, comenzando por el bit inmediatamente a la derecha de la coma. El procedimiento es el siguiente:

$$\begin{aligned} 0.375 \times 2 &= \mathbf{0.75} \longrightarrow b_{-1} = 0 \\ 0.75 \times 2 &= \mathbf{1.5} \longrightarrow b_{-2} = 1 \\ 0.5 \times 2 &= \mathbf{1.0} \longrightarrow b_{-3} = 1 \end{aligned}$$

Luego, debemos normalizar el resultado desplazando el punto binario:

$$(11.011)_2 \rightarrow (1.1011)_2 \times 2^1$$

Por lo tanto:

$$N = (3.375)_{10} = (-1)^0 \times (1.1011)_2 \times 2^1$$

Notar que el significando $m = (1.1011)_2$ cumple la condición de normalización $1 \leq m < 2$, ya que su parte entera es igual a 1 y la parte fraccionaria es menor que 1. Esta propiedad es característica de la representación normalizada en base 2 y garantiza una representación única del número.

Observación

Al realizar la normalización se debe tener en cuenta la corrección del exponente.

3.2. Características de los números en notación científica normalizada

Sea el conjunto $\mathbb{F}(\beta, t, L, U)$ de números en notación científica normalizada, donde t es la cantidad de dígitos en la parte fraccionaria del significando, y L y U son las cotas inferior y superior del exponente e , respectivamente, con $-L \leq e \leq U$. Entonces:

1. El número de elementos del conjunto $\mathbb{F}$ es

$$2 \cdot (\beta - 1) \cdot \beta^t \cdot (U + L + 1)$$

Esta expresión se obtiene contando las posibles combinaciones de sus componentes:

- **Signo:** hay 2 posibilidades (+ y -).
- **Exponente:** el exponente e puede tomar valores enteros entre $-L$ y U , por lo que hay $(U + L + 1)$ posibilidades.
- **Significante:** en un sistema normalizado, el primer dígito debe ser distinto de cero, es decir, puede tomar $(\beta - 1)$ valores posibles. Los restantes t dígitos pueden tomar β valores cada uno, por lo que el número total de significantes es

$$(\beta - 1)\beta^t.$$

Multiplicando todas las posibilidades se obtiene:

$$2 \cdot (\beta - 1)\beta^t \cdot (U + L + 1).$$

2. El número cero no puede representarse en forma normalizada.
3. Si $x \in \mathbb{F}$, entonces su opuesto $-x \in \mathbb{F}$.
4. El conjunto está acotado superior e inferiormente. En particular,

$$x_{\min} \leq |x| \leq x_{\max},$$

donde

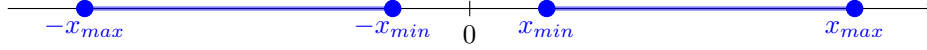
- $x_{\min} = \beta^{-L}$,
- $x_{\max} = (\beta - \beta^{-t})\beta^U$.

Obsérvese que no existen números representables en el intervalo $(-x_{\min}, x_{\min})$, lo que genera una zona de subrepresentación alrededor del cero.

5. Existen intervalos de la recta real que no pueden representarse en $\mathbb{F}$:
 - $(-\infty, -x_{\max})$ (*overflow* negativo).
 - $(-x_{\min}, 0)$ (*underflow* negativo).
 - $(0, x_{\min})$ (*underflow* positivo).

- $(x_{\text{máx}}, +\infty)$ (*overflow* positivo).

Además, el número 0 se trata por separado y suele incorporarse explícitamente al conjunto $\mathbb{F}$.



- Los números en el conjunto $\mathbb{F}$ no están uniformemente distribuidos sobre la recta real: son más densos cerca del origen y más espaciados a medida que aumenta su magnitud (este comportamiento se analizará en detalle en la Sección 3.4).

3.3. Rango y precisión en la representación en punto flotante

La representación en punto flotante permite representar números reales mediante una estructura normalizada de la forma:

$$x = (-1)^s \times m \times \beta^e,$$

donde s es el bit de signo, m es el significando normalizado, e es el exponente y β es la base del sistema.

Por lo tanto, el rango de valores representables está determinado principalmente por el número de bits asignados al exponente. Dado que el significando satisface

$$1 \leq m < \beta,$$

el mayor número representable (normalizado) es de la forma

$$|x_{\text{máx}}| = (\beta - \beta^{-t}) \times \beta^{e_{\text{máx}}},$$

donde t es el número de dígitos en la parte fraccionaria del significando en base β .

De manera análoga, el menor número positivo normalizado es

$$|x_{\text{mín}}| = \beta^{e_{\text{mín}}}.$$

Además, veremos que en los sistemas que soportan números subnormales, es posible representar valores aún menores, sacrificando la normalización del significando, lo que permite extender el rango hacia valores cercanos a cero.

Por otra parte, la precisión de la representación está determinada por la cantidad de dígitos del significando. En particular, si el significando tiene t dígitos en la parte fraccionaria, entonces la separación entre números consecutivos representables es

$$\Delta = \beta^{-t} \times \beta^e.$$

Esto implica que la separación absoluta entre números representables crece con el valor del exponente. En consecuencia, la representación en punto flotante presenta una distribución no uniforme: los números están más densamente representados cerca de cero y más espaciados a medida que aumenta su magnitud.

Observaciones

- El **rango** determina el intervalo de magnitudes representables, mientras que la **precisión** determina la resolución (espaciado) entre números representables. Dado que se dispone de una cantidad finita de bits, existe un compromiso entre rango y precisión: aumentar el rango (más bits para el exponente) reduce la precisión (menos bits para la mantisa), y viceversa.
- La representación en punto flotante presenta una **densidad no uniforme**: la separación entre números representables crece a medida que aumenta el exponente. Esto explica fenómenos como la pérdida de significancia y la cancelación catastrófica en operaciones numéricas, que se analizarán más adelante.

3.4. Distribución no uniforme

Al contrario que con los números en punto fijo, en punto flotante la distribución en la recta numérica no es uniforme y a medida que nos alejamos del origen se van separando. Supongamos que el número de bits de la parte fraccionaria del significante sea t . En el intervalo $[2^e, 2^{e+1})$ es posible representar 2^t números equiespaciados y separados con una distancia $2^e/2^t$.

La Figura 1 ilustra esta idea, donde se puede observar como de manera general un número real quedará ubicado entre dos números representables con un error de representación acotado por la mitad de la distancia entre

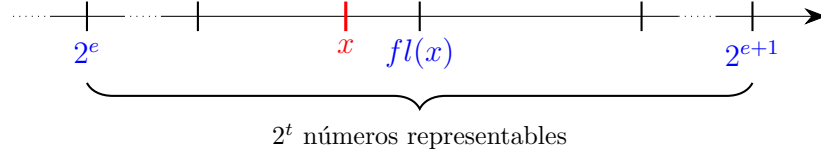


Figura 1: Error de representación de números reales. El valor real x se aproxima mediante $fl(x)$, que representa el valor más cercano dentro del conjunto $\mathbb{F}$ de números representables.

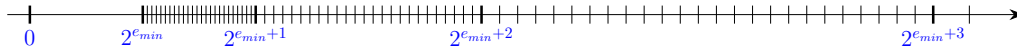


Figura 2: Esquema mostrando la distribución no uniforme sobre la recta real (solo parte positiva).

dichos números. Por ejemplo, si $t = 23$, entre $2^{20} = 1048576$ y $2^{21} = 2097152$ hay $2^{23} = 8388608$ números pero el espaciado es de solo 0.125.

De este modo se deriva una regla práctica que cuando es necesario comparar dos números en punto flotante relativamente grandes es preferible comparar la diferencia relativa entre esos dos números en lugar de las magnitudes absolutas de los mismos dado que la precisión es menor cuanto más grandes sean los números.

La Figura 2 muestra esquemáticamente la disposición de los números representables en punto flotante sobre la recta real. Puede observarse que la densidad es mayor cerca del cero y decrece al aumentar el valor absoluto. Asimismo, en la Figura 3 se puede apreciar de manera numérica cómo aumenta la separación entre dos números consecutivos en función del exponente e .

3.5. Épsilon de máquina

Una cantidad de gran importancia es la denominada **épsilon de máquina** (ϵ_m), que representa la distancia entre el número 1 y el siguiente número mayor en punto flotante. Es decir, ϵ_m es el menor número tal que $1 + \epsilon_m > 1$. Por lo tanto, teniendo en cuenta la estructura de los números en formato punto flotante se puede calcular que:

$$\boxed{\epsilon_m = 2^{-t}},$$

donde t es la cantidad de bits de la parte fraccionaria del significando. A esta expresión se puede llegar teniendo en cuenta que $(1)_{10} = 2^0$ y el siguiente número representables es $(1 + 2^{-t}) \times 2^0$.

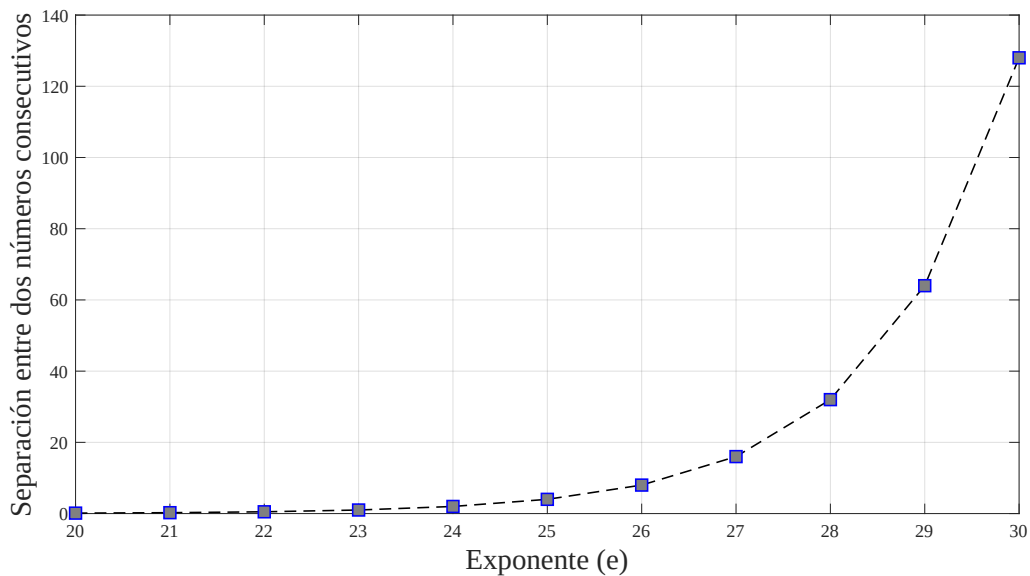


Figura 3: Evolución de la separación entre dos números consecutivos en punto flotante en función del exponente e , para el intervalo $20 \leq e \leq 30$.

¿Por qué es importante el **épsilon de máquina**? Su importancia radica en que cuantifica la precisión de la representación en punto flotante y permite acotar el error de redondeo, como se verá en la Sección 6.4.

3.6. Ventajas y limitaciones de la representación en punto flotante

La representación en punto flotante constituye el estándar para el tratamiento de números reales en computación, ya que permite manejar magnitudes muy diversas dentro de un mismo esquema. Sin embargo, como toda representación finita, implica ciertos compromisos.

Ventajas

- **Gran rango dinámico:** permite representar números muy grandes y muy pequeños mediante el uso de un exponente.
- **Precisión relativa aproximadamente constante:** el error relativo se mantiene acotado en gran parte del rango de representación.

- **Adaptabilidad a distintas magnitudes:** resulta adecuada cuando los datos varían en varios órdenes de magnitud.

Limitaciones

- **Errores de redondeo:** la mayoría de los números reales no pueden representarse exactamente, lo que introduce errores inevitables.
- **Pérdida de propiedades algebraicas:** operaciones como la asociatividad o la distributividad no se cumplen en general debido al redondeo.
- **Sensibilidad numérica:** ciertas operaciones, como la resta de números cercanos, pueden amplificar errores (cancelación).
- **Distribución no uniforme:** la separación entre números representables depende de la magnitud.
- **Mayor complejidad aritmética:** las operaciones requieren etapas adicionales (alineación de exponentes, normalización y redondeo), así como el tratamiento de valores especiales.

En este sentido, el punto flotante establece un equilibrio entre rango dinámico y precisión, a costa de una mayor complejidad y de la introducción de errores de redondeo que deben ser tenidos en cuenta en el análisis numérico.

4. Estándar IEEE 754 para números en punto flotante

Para evitar la proliferación de diversos sistemas de punto flotante, el Instituto de Ingenieros Eléctricos y Electrónicos (IEEE) creó un comité que en 1985 estableció una norma unificada para la representación y el manejo de números en punto flotante: el **estándar IEEE 754**.

Esta norma no solo define formatos de representación (como simple y doble precisión), sino también reglas para las operaciones aritméticas, modos de redondeo y el tratamiento de situaciones excepcionales, tales como overflow, underflow, división por cero y resultados indefinidos. Asimismo, introduce conceptos fundamentales como los números subnormales, los valores especiales ($\pm\infty$, NaN) y garantiza propiedades importantes como la reproducibilidad y la portabilidad de los cálculos numéricos entre distintas arquitecturas.

4.1. Representación de números normalizados en IEEE 754

Un **número normalizado** según el estándar IEEE 754 se representa, en notación científica binaria normalizada, de la siguiente manera:

$$N = (-1)^s \times (1.f) \times 2^e \quad (1)$$

donde:

- s indica el **signo** del número, con la convención:

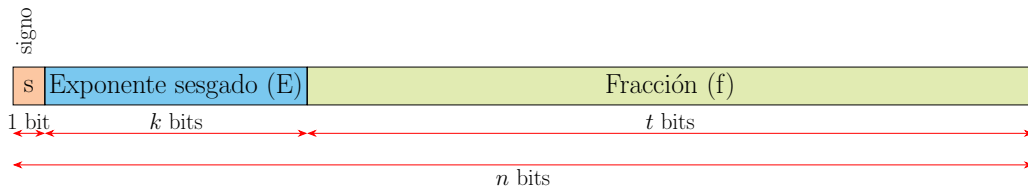
$$s = \begin{cases} 0 & \text{si el número es positivo,} \\ 1 & \text{si el número es negativo.} \end{cases} \quad (2)$$

- e es el **exponente real** (no sesgado), tal que $e_{\min} \leq e \leq e_{\max}$.
- $(1.f)$ es el **significando** (o mantisa), donde f representa la parte fraccionaria, definido como:

$$(1.f) = (1.b_{-1} \cdots b_{-t})_2 = 1 + b_{-1} 2^{-1} + \cdots + b_{-t} 2^{-t},$$

siendo t la cantidad de bits de la parte fraccionaria del significando.

La representación a nivel bits de un número de punto flotante formato IEEE 754 se divide en tres campos de bits a partir de la Ecuación (1):



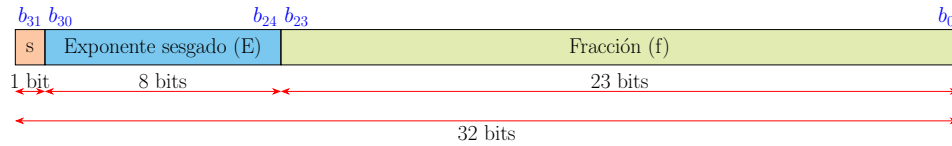
- **Bit de signo (s):** El bit más significativo codifica directamente el signo s , de acuerdo con la convención expresada en la Ecuación (2).
- **Bits del exponente sesgado (E):** Los siguientes k bits codifican el exponente sesgado E . En la Sección 4.3 veremos en detalle el concepto de exponente sesgado.
- **Bits de la fracción del significando (f):** Los últimos t bits codifican la parte fraccionaria del significando $(1.f)$. En la Sección 4.4 veremos en detalle el concepto de significando.

4.2. Formatos IEEE 754

La norma IEEE 754 define varios formatos de punto flotante, incluyendo cuatro formatos binarios básicos (16, 32, 64 y 128 bits) y tres formatos decimales, además de formatos extendidos cuya implementación depende de la arquitectura. En este curso se consideran los formatos binarios más utilizados: simple (32 bits), doble (64 bits) y cuádruple precisión (128 bits).

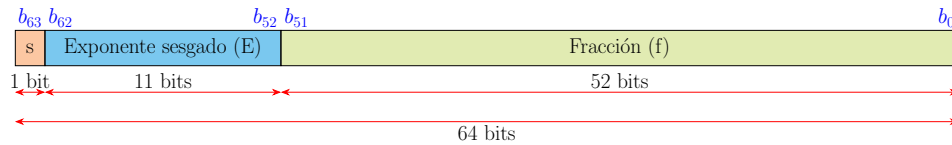
1. Precisión simple

Los números en precisión simple tienen 32 bits con la siguiente distribución de bits:



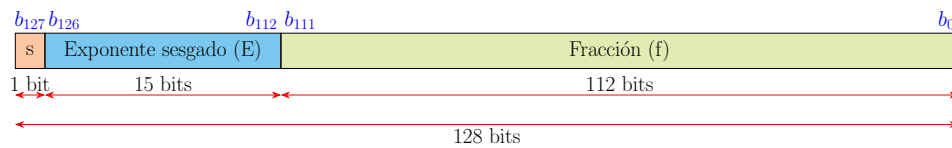
2. Precisión doble

Los números en precisión doble tienen 64 bits con la siguiente distribución de bits:



3. Precisión cuádruple

Los números en precisión cuádruple tienen 128 bits con la siguiente distribución de bits:



4.3. Exponente sesgado

La norma IEEE 754 almacena un exponente con k bits, denominado **exponente sesgado**

$$E = (b_{k-1} \cdots b_0)_2,$$

cuyo rango es $[0, 2^k - 1]$ y que se relaciona con el exponente e en la Ecuación (1) de la siguiente manera:

$$e = E - \text{sesgo},$$

donde el **sesgo** vale $2^{k-1} - 1$.

El exponente almacenado E es siempre no negativo, ya que se representa mediante un sesgo (*bias*). Esto permite que el exponente real e pueda tomar tanto valores positivos como negativos, recuperándose a partir de E mediante la resta del sesgo. La Figura 4 ilustra esta idea para el caso del formato de simple precisión.

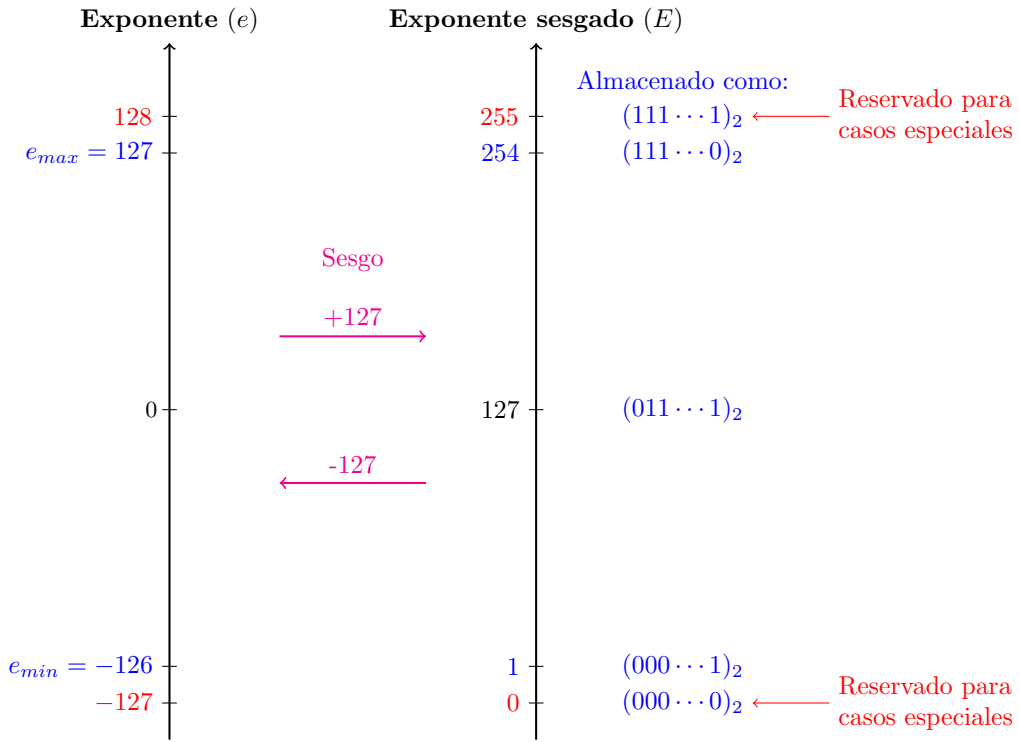


Figura 4: Representación con exponente sesgado para simple precisión.

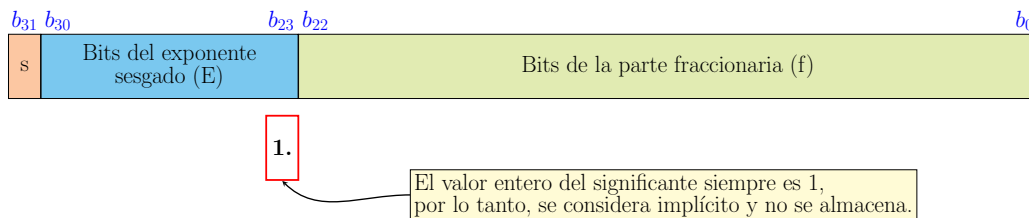
Los valores mínimos ($E = 0$) y máximo del exponente sesgado ($E = 255$, $E = 2047$ y $E = 32767$, según el formato) no se utilizan para números normalizados sino que tienen usos especiales como se verá más adelante. Por lo tanto, los valores mínimos y máximos de exponente para números normalizados en la Ecuación (1) resultan:

- Precisión simple:

- Cantidad de bits: $k = 8$
 - Sesgo: $2^{k-1} - 1 = 127$
 - $e_{min} = 1 - 127 = -126$
 - $e_{max} = 254 - 127 = 127$
- Precisión doble:
- Cantidad de bits: $k = 11$
 - Sesgo: $2^{k-1} - 1 = 1023$
 - $e_{min} = 1 - 1023 = -1022$
 - $e_{max} = 2046 - 1023 = 1023$
- Precisión cuádruple:
- Cantidad de bits: $k = 15$
 - Sesgo: $2^{k-1} - 1 = 16383$
 - $e_{min} = 1 - 16383 = -16382$
 - $e_{max} = 32766 - 16383 = 16383$

4.4. Significando

La definición de notación científica presentada en la Sección 3.1 indica que el significante se compone de un bit igual a uno (parte entera), seguido de un punto y luego el resto de los bits del significante (23, 52 o 112 bits, según el formato), que constituyen la parte fraccionaria. Entonces, ese bit, que siempre es igual a 1, no necesita ser almacenado, ya que se puede asumir su presencia. La siguiente figura muestra esta idea para el formato simple precisión:



Si todos los bits del significante son ceros, su magnitud es 1. En cambio, si todos los bits son unos, el valor resultante es apenas inferior a 2:

$$1 \leq (1.f) < 2 - 2^{-t}$$

Observación

Notar que esta normalización en principio deja sin representación al cero porque $1 \leq (1.f)$. Sin embargo, en la Sección 4.7.2 veremos una de las excepciones a esta regla y que por lo tanto el número cero se puede representar como caso especial.

4.5. Conversión de decimal a IEEE 754

A continuación se presenta el procedimiento de conversión de un número decimal al formato IEEE 754 mediante un ejemplo en simple precisión. Para otras precisiones, el procedimiento es análogo, con los ajustes correspondientes en el sesgo y en el número de bits.

Ejemplo

Se quiere convertir el número decimal $N = (2.625)_{10}$ a formato IEEE 754 simple precisión. El procedimiento se compone de los siguientes pasos:

1. Convertir la parte entera a binario: $(2)_{10} = (10)_2$
2. Convertir la parte fraccional a binario:

$$\begin{array}{rclcl} 0.625 \times 2 & = & 1.25 & \longrightarrow & b_{-1} = 1 \\ 0.25 \times 2 & = & 0.5 & \longrightarrow & b_{-2} = 0 \\ 0.5 \times 2 & = & 1.0 & \longrightarrow & b_{-3} = 1 \end{array}$$

Por lo tanto:

$$(0.625)_{10} = (0.101)_2$$

3. Ya tenemos el número convertido a binario:

$$N = (2.625)_{10} = (10.101)_2$$

4. Agregar el exponente: $N = (10.101)_2 = (10.101)_2 \times 2^0$
5. Normalizar según lo visto previamente:

$$N = (10.101)_2 \times 2^0 = (1.0101)_2 \times 2^1$$

6. Por lo tanto, el significante resulta $(1.010100000000000000000000)_2$ donde el primer 1 y el punto están implícitos, es decir no se almacenan.

7. Corregir el exponente sumando el sesgo correspondiente ($sesgo = 127$):
 $E = 1 + 127 = 128$
8. Convertir el exponente a binario: $E = (128)_{10} = (10000000)_2$
9. El número es positivo. Por lo tanto el bit de signo es $s = 0$.
10. Finalmente, el número $N = (2.625)_{10}$ convertido a IEEE 754 simple precisión resulta:

$$\underbrace{0}_S \quad \underbrace{10000000}_{\text{exponente sesgado}} \quad \underbrace{010100000000000000000000}_{\text{fracción del significante}}$$

A esta secuencia de bits la podemos expresar como $(0|10000000|010100000000000000000000)_2$, utilizando el símbolo $|$ como separador visual para los tres campos del números.

11. Es usual representar esta secuencia de bits en formato hexadecimal para tener una forma más compacta de representación. Por lo tanto, $N = 0x40280000$.

4.6. Conversión de IEEE 754 a decimal

A continuación mostramos el procedimiento de conversión de formato IEEE 754 a decimal mediante un ejemplo en simple precisión. Para otras precisiones el procedimiento es análogo con los cambios necesarios en el sesgo y el número de bits.

Ejemplo

Se quiere convertir el número expresado en IEEE 754 simple precisión $N = (0|10000010|101101000000000000000000)_2$ a decimal. El procedimiento consiste en los siguientes pasos:

1. Separar el número en sus diferentes partes:

Signo	Exponente sesgado	Fracción del significante
1 bit	8 bits	23 bits
0	10000010	101101000000000000000000

2. El bit de signo es cero por lo tanto el número es positivo.

Tabla 1: Clases de valores en IEEE 754 y su representación.

Signo	Exponente	Fracción	Representa	Denominación
$s = 0/1$	$e = e_{min} - 1$ $E = (00 \dots 0)_2$	$f = 0$	± 0	Ceros
		$f \neq 0$	$(-1)^s \times (0.f) \times 2^{e_{min}}$	Núm. denormal.
	$e_{min} \leq e \leq e_{max}$ $E = (b_{k-1} \dots b_0)_2$	$0 \leq f < 1$	$(-1)^s \times (1.f) \times 2^e$	Núm. normaliz.
	$e = e_{max} + 1$ $E = (11 \dots 1)_2$	$f = 0$	$\pm \infty$	Infinitos
		$f \neq 0$	$\pm \text{NaN}$	<i>Not a Number</i>

3. Convertir el exponente a decimal:

$$E = (10000010)_2 = (130)_{10}$$

4. Restar al exponente el sesgo correspondiente ($sesgo = 127$):

$$e = 130 - 127 = 3$$

5. Convertir el significando a decimal:

$$(1.101101000000000000000000)_2 = 1 + 1 \times 2^{-1} + 1 \times 2^{-3} + 1 \times 2^{-4} + 1 \times 2^{-6} = (1.703125)_{10}$$

6. Finalmente, el número convertido a decimal resulta:

$$N = (-1)^0 \times 1.703125 \times 2^3 = (13.625)_{10}$$

4.7. Valores especiales en punto flotante (IEEE 754)

La norma IEEE 754 va más allá de la definición de formatos, especificando reglas de representación y de aritmética que garantizan resultados uniformes y predecibles entre plataformas. En este contexto, además de los números normalizados, el estándar define un conjunto de valores especiales —denormalizados (o subnormales), ceros con signo, infinitos y NaN— que completan el conjunto de números representables dentro de cada formato. La Tabla 1 resume las clases de valores representables en IEEE 754 según el patrón de bits.

4.7.1. Números denormalizados

El menor número normalizado en simple precisión es 1.0×2^{-126} . Antes de la definición del estándar si se presentaban problemas de “*underflow*” debía redondearse a cero. Los **números denormalizados** (también denominados “subnormales”) del estándar tienen una mejor aproximación al problema. Estos números tienen un exponente E igual a cero (no permitido para los números normalizados) y una fracción de 23, 52 o 112 bits, según la precisión, distinta de cero. El “1” implícito del significante es cero para los números denormalizados. Por lo tanto, el significante de los números denormalizados es mayor a cero y menor a uno. Por lo tanto, la representación para números denormalizados se puede formalizar de la siguiente manera:

$$N = (-1)^s \times (0.f) \times 2^{e_{min}}$$

donde $e_{min} = -126$ para simple precisión, $e_{min} = -1022$ para doble precisión y $f \neq 0$.

De esta manera se pueden representar números más pequeños que los números normalizados. Por ejemplo, trabajando en simple precisión el rango que cubren los números desnormalizados (que se suma al rango de los números normalizados) es $[2^{-23} \times 2^{-126}, (1 - 2^{-23}) \times 2^{-126}]$. Por lo tanto, el menor número representable en simple precisión es 2^{-149} , menor que el menor número normalizado (2^{-126}).

En la Figura 5 se observa el desbordamiento a cero gradual (“*gradual underflow*”) introduciendo los números de punto flotante desnormalizados.

Observación

Notar que en la representación de los números denormalizados el exponente en la representación es $e = e_{min}$ y no $e = e_{min} - 1$. Es decir, el exponente del número representado es $e = -126$ para simple precisión ($e = -1022$ para doble precisión) a pesar de que en el registro realmente se almacena el exponente sesgado $E = (0000\ 0000)_2$ que correspondería al exponente $e = e_{min} - 1 = -127$ ($e = -1023$ para doble precisión). De lo contrario quedaría un “hueco” entre 2^{-127} y 2^{-126} .

4.7.2. Ceros

Un caso especial dentro del conjunto de los números denormalizados es el número cero. El cero se representa por el exponente $e_{min} - 1$ (es decir,

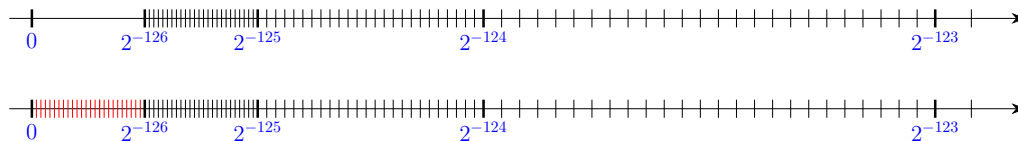


Figura 5: Desbordamiento a cero gradual mediante números de punto flotante denormalizados. Ejemplo con simple precisión: en la recta real inferior se observa cómo el rango entre 0 y 2^{-126} se divide en 2^{23} valores.

se codifica con un exponente sesgado con todos ceros: $E = (0 \dots 0)_2$ y un significante cero (es decir, todos ceros en la parte fraccionaria). Dado que el bit de signo puede tomar dos valores diferentes, hay dos ceros, $+0$ y -0 . Con el formato de punto flotante IEEE 754, los valores -0.0 y $+0.0$ se consideran diferentes en algunos aspectos e iguales en otros.

4.7.3. Infinitos

El estándar IEEE 754 provee una representación para los números infinitos ($\pm\infty$), los cuales pueden aparecer como resultado de ciertas operaciones aritméticas y ser utilizados como operandos en cálculos posteriores.

Ejemplos

A continuación se enumeran algunas operaciones que dan como resultado un valor de tipo infinito:

1. $N + (+\infty) = +\infty$
2. $N - (+\infty) = -\infty$
3. $N + (-\infty) = -\infty$
4. $N - (-\infty) = +\infty$
5. $(+\infty) + (+\infty) = +\infty$
6. $(-\infty) + (-\infty) = -\infty$
7. $(-\infty) - (+\infty) = -\infty$
8. $(+\infty) - (-\infty) = +\infty$

4.7.4. Valor NaN

El valor NaN (*Not a Number*) es una representación especial en aritmética de punto flotante. Se utiliza para indicar resultados indefinidos o no representables, como los que surgen en operaciones inválidas (por ejemplo, $0/0$ o $\infty - \infty$), y también puede emplearse para señalar variables no inicializadas o datos inválidos.

Ejemplos

A continuación se enumeran algunas operaciones que dan como resultado un valor de tipo NaN:

1. $0/0 = \text{NaN}$
2. $\pm\infty / \pm\infty = \text{NaN}$
3. $0 \times \pm\infty = \text{NaN}$
4. $\infty - \infty = \text{NaN}$

Ejemplo

Ejemplo usando la cabecera `math.h`:

```
#include <stdio.h>
#include <math.h>

int main(){
    float a=NaN, b=INFINITY;
    printf("%f %f\n",a,b);
    return 0;
}
```

Ejemplo

Usando la cabecera `ieee754.h`:

```
#include <stdio.h>
#include <math.h>
#include <ieee754.h>
```

```

int main(){
    union ieee754_float myfloat;
    myfloat.f = INFINITY;
    printf("Signo: %x\n", myfloat.ieee.negative);
    printf("Exponente: %x\n", myfloat.ieee.exponent);
    printf("Mantisa: %x\n", myfloat.ieee.mantissa);

    union ieee754_double mydouble;
    mydouble.d = NAN;
    printf("Signo: %x\n", mydouble.ieee.negative);
    printf("Exponente: %x\n", mydouble.ieee.exponent);
    printf("Mantisa (b51 a b32): %x\n", mydouble.ieee.
        mantissa0);
    printf("Mantisa (b31 a b0): %x\n", mydouble.ieee.
        mantissa1);
    return 0;
}

```

4.8. Rango y precisión en la representación IEEE 754

En el estándar IEEE 754, los números en punto flotante se representan en base $\beta = 2$, con una estructura de la forma

$$x = (-1)^s \times m \times 2^e,$$

donde s es el bit de signo, m es el significando y e es el exponente.

Para los números normalizados, el significando tiene la forma

$$m = 1.f,$$

donde f es la parte fraccionaria codificada en t bits. Por lo tanto,

$$1 \leq m < 2.$$

El exponente satisface $e \in [e_{\min}, e_{\max}]$. En consecuencia, el mayor número normalizado representable es

$$|x_{\max}^{\text{norm.}}| = (2 - 2^{-t}) \times 2^{e_{\max}},$$

mientras que el menor número positivo normalizado es

$$|x_{\min}^{\text{norm.}}| = 2^{e_{\min}}.$$

Además, IEEE 754 introduce números subnormales (o denormalizados), en los cuales el significando no está normalizado:

$$m = 0.f.$$

Esto permite representar valores más pequeños que $|x_{\min}|$, siendo el menor positivo representable

$$|x_{\min}^{\text{sub.}}| = 2^{e_{\min}} \times 2^{-t}.$$

Por otra parte, la precisión está determinada por la cantidad de bits del significando. Si el significando tiene t bits en la fracción, entonces la separación entre números consecutivos representables para un exponente e es

$$\Delta = 2^{e-t}.$$

Esto implica que la separación absoluta entre números representables crece con el valor del exponente. En consecuencia, la representación en punto flotante presenta una distribución no uniforme: los números están más densamente representados cerca de cero y más espaciados a medida que aumenta su magnitud.

Observación

El rango determina qué valores pueden representarse, mientras que la precisión determina cuán densamente están distribuidos esos valores dentro del rango. En IEEE 754, esta densidad es no uniforme: la separación entre números representables crece exponencialmente con el exponente. Esto explica fenómenos como la pérdida de significancia y la cancelación catastrófica en operaciones numéricas.

Observación

En la Sección 3.5 se definió el épsilon de máquina (ϵ_m) como una medida de la precisión en punto flotante. En IEEE 754:

- En **simple precisión** ($t = 23$ bits de fracción), se tiene:

$$\epsilon_m = 2^{-23} \approx 1.19 \times 10^{-7}.$$

- En **doble precisión** ($t = 52$ bits de fracción), se tiene:

$$\epsilon_m = 2^{-52} \approx 2.22 \times 10^{-16}.$$

5. Operaciones con números en punto flotante

5.1. Suma/resta

Para sumar o restar números en punto flotante es necesario igualar sus exponentes, desplazando el significante del número con menor exponente. Una vez alineados, se pueden sumar o restar los significantes. Concretamente, la secuencia de pasos es la siguiente:

1. Verificar NaN: si alguno de los operandos es NaN, el resultado es NaN.
2. Verificar INF - INF: en este caso, el resultado es NaN.
3. Verificar +INF: si alguno de los operandos es +INF, el resultado es +INF.
4. Verificar -INF: si alguno de los operandos es -INF, el resultado es -INF.
5. Verificar ceros: si uno de los operandos es 0, el resultado es el otro operando ($X + 0 = X$).
6. Verificar diferencia de exponentes: si la diferencia entre los exponentes es mayor que el número de bits del significante, el resultado es el operando de mayor magnitud, ya que el menor no afecta al resultado.
7. Igualar exponentes: para realizar la suma o resta, los exponentes deben coincidir. Se desplaza hacia la derecha el significante del número con menor exponente hasta igualarlos.
8. Sumar o restar los significantes.
9. Normalizar el resultado (si es necesario): el resultado debe expresarse en forma normalizada, es decir, con un significante de la forma $1.b_{-1}b_{-2}\dots$. Si se ajusta el punto binario, debe modificarse el exponente en consecuencia.

Ejemplo

Supongamos que queremos hacer la suma

$$S = 123 + 456 = 579$$

El número 123 se puede expresar como

$$(123)_{10} = (-1)^0 \times 2^6 \times 1.921875$$

que a su vez equivale a la secuencia de bits en formato IEEE 754

$$(0|10000101|111011000000000000000000)_2$$

Por otra parte, el número 456 se puede expresar como

$$(456)_{10} = (-1)^0 \times 2^8 \times 1.78125$$

que equivale a la secuencia de bits

$$(0|10000111|110010000000000000000000)_2$$

Como ambos números tienen diferentes exponentes lo primero que hay que hacer es igualar los exponentes (el menor hacia el mayor) ajustando al mismo tiempo los significantes. En este ejemplo tenemos que correr la coma dos lugares hacia a izquierda. Luego podemos hacer la suma de los significantes:

$$\begin{array}{r} 1.110010000000000000000000 \\ + \quad 0.011110110000000000000000 \\ \hline 10.010000110000000000000000 \end{array}$$

Una vez realizada la suma debemos ajustar la coma y el exponente para que el significante quede normalizado:

$$\text{Significante} = (1.001000011000000000000000)_2$$

y el exponente de la suma resulta $e = 9$. Por lo tanto, el resultado es:

$$S = (-1)^0 \times 2^9 \times 1.130859375 = 579$$

que equivale a la secuencia de bits

$$(0|10001000|001000011000000000000000)_2$$

Este resultado también lo podemos expresar usando notación hexadecimal para que quede más compacto: **0x4410c000**.

5.2. Multiplicación

Para multiplicar números en punto flotante, se multiplican los significantes y se suman los exponentes. Concretamente, la secuencia de pasos es la siguiente:

1. Verificar NaN: si alguno de los operandos es NaN, el resultado es NaN.
2. Verificar 0×INF: si un operando es 0 y el otro es INF, el resultado es NaN.
3. Verificar ceros: si alguno de los operandos es 0, el resultado es 0.
4. Sumar los exponentes.
5. Multiplicar los significantes.
6. Normalizar el producto (si es necesario).

Ejemplo

Supongamos que queremos hacer la multiplicación $N = 120 \times 12 = 1440$ en norma IEEE 754. Primero convertimos ambos números:

$$\begin{aligned}(120)_{10} &= 2^6 \times 1.875 = (0|10000101|1110000000000000000000)_2 \\ (12)_{10} &= 2^3 \times 1.5 = (0|10000010|1000000000000000000000)_2\end{aligned}$$

Luego podemos multiplicar los significantes y sumar los exponentes. La suma de los exponentes es $e = 9$, y el producto de los significantes se calcula de la siguiente manera:

$$\begin{array}{r} 1. \ 1 \ 1 \ 1 \ 0 \ \dots \\ \times \\ 1. \ 1 \ 0 \ 0 \ 0 \ \dots \\ \hline 1 \ 1 \ 1 \ 1 \\ + \\ 1 \ 1 \ 1 \ 1 \\ \hline 1 \ 0. \ 1 \ 1 \ 0 \ 1 \end{array}$$

Como el significante no está normalizado, es necesario normalizarlo aumentando en uno el exponente y corriendo un lugar el punto del significante. Por lo tanto, el resultado de la multiplicación es:

$$N = 2^{10} \times (1.011010000000000000000000)_2 = 2^{10} \times 1.40625 = (1440)_{10}$$

que equivale a la secuencia de bits $(0|10001001|0110100000000000000000)_2$ o a **0x44b40000** en notación hexadecimal.

5.3. División

Para dividir números en punto flotante, se dividen los significantes y se restan los exponentes. Concretamente, la secuencia de pasos es la siguiente:

1. Verificar NaN: si alguno de los operandos es NaN, el resultado es NaN.
2. Verificar 0/0: si ambos operandos son 0, el resultado es NaN.
3. Verificar INF/INF: si ambos operandos son INF, el resultado es NaN.
4. Verificar INF/X: si el dividendo es INF y $X \neq 0$, el resultado es INF.
5. Verificar X/INF: si el divisor es INF, el resultado es 0.
6. Verificar ceros: si el divisor es 0, el resultado es INF con el signo correspondiente. Si el dividendo es 0 y el divisor es distinto de 0, el resultado es 0.
7. Restar los exponentes.
8. Dividir los significantes.
9. Normalizar el resultado (si es necesario).

Ejemplo

Supongamos que queremos hacer la división $D = N_1/N_2 = 96/15 = 6.4$. Primero convertimos ambos números:

$$\begin{aligned}(96)_{10} &= 2^6 \times 1.5 = (0|10000101|100000000000000000000000)_2 \\ (15)_{10} &= 2^3 \times 1.875 = (0|10000010|111000000000000000000000)_2\end{aligned}$$

Luego podemos dividir los significantes y restar los exponentes:

$$D = 2^3 \times 0.8$$

Como el significativo no está normalizado, es necesario normalizarlo disminuyendo en uno el exponente y corriendo un lugar el punto del significativo. Notar que correr el punto binario un lugar hacia la derecha equivale a multiplicar por dos su equivalente en decimal. Por lo tanto, el resultado de la división es:

$$M = 2^2 \times 1.6 = (6.4)_{10}$$

En realidad, el número 6.4 no tiene representación exacta y el número más cercano es 6.400000095367431640625 que equivale a la secuencia de bits $(0|10000001|10011001100110011001101)_2$ o a 0x40cccccd en notación hexadecimal.

6. Error en la representación de números reales

6.1. Representación finita en IEEE 754

Los sistemas de cómputo representan números reales mediante una cantidad finita de bits. En particular, vimos que el estándar IEEE 754 define la representación en punto flotante como:

$$x = (-1)^s \times m \times \beta^e,$$

donde el significando m tiene un número finito de dígitos.

Debido a esta limitación:

- No todos los números reales pueden representarse exactamente.
- Muchos números racionales tienen expansión infinita en base $\beta = 2$.

Por ejemplo,

$$(0.1)_{10} = (-1)^0 \times (1.1001100110011 \dots)_2 \times 2^{-4}.$$

Por lo tanto, en memoria se almacena una aproximación del número, ya sea por redondeo o por truncamiento.

6.2. Redondeo de un número real

En general, cualquier número real (x) debe aproximarse por un elemento del conjunto de números representables $\mathbb{F}$, lo cual se denota por $fl(x)$. La forma usual de proceder consiste en aplicar el **redondeo al más cercano** (también llamado redondeo simétrico) a t dígitos en la parte fraccionaria del significando. Es decir, si

$$x = (-1)^s \times (a_0.a_{-1}a_{-2} \dots a_{-t}a_{-t-1} \dots)_\beta \times \beta^e,$$

entonces su representación en máquina se define como

$$fl(x) = (-1)^s \times (a_0.a_{-1}a_{-2} \dots \tilde{a}_{-t})_\beta \times \beta^e,$$

donde el dígito $\tilde{a}_{-t}$ se obtiene según la regla de redondeo:

$$\tilde{a}_{-t} = \begin{cases} a_{-t}, & \text{si } a_{-t-1} < \beta/2, \\ a_{-t} + 1, & \text{si } a_{-t-1} \geq \beta/2. \end{cases}$$

En caso de que $a_{-t} + 1 = \beta$, se produce un acarreo que debe propagarse hacia los dígitos más significativos.

El error introducido por este proceso se denomina **error de redondeo**.

Ejemplos

Supongamos que queremos redondear con 5 dígitos significativos:

$$\begin{aligned} N_1 &= (4.567689010\dots)_{10} \rightarrow (4.5677)_{10} \rightarrow \mathbf{error} \approx 6.9 \times 10^{-4} \\ N_2 &= (1.10101010\dots)_2 \rightarrow (1.1011)_2 \rightarrow \mathbf{error} = 0.0234 \end{aligned}$$

6.3. Redondeo en IEEE 754

Dado que el conjunto de números representables $\mathbb{F}$ es finito, cualquier operación produce un resultado que debe redondearse.

El estándar IEEE 754 define varios modos de redondeo:

- Redondeo al más cercano (modo por defecto)
- Redondeo hacia cero
- Redondeo hacia $+\infty$
- Redondeo hacia $-\infty$

Ejemplo

Si siguiendo con el caso particular del número $(0.1)_{10}$, si se redondea al número más cercano, se obtiene:

$$(0.1)_{10} \approx (-1)^0 \times (1.10011001100110011001101)_2 \times 2^{-4}.$$

En formato de precisión simple (IEEE 754), el valor efectivamente almacenado es aproximadamente 0.10000000149.

6.4. Error de representación

Sea $x \in \mathbb{R}$ y $fl(x) \in \mathbb{F}$ su representación en máquina.

- **Error absoluto:**

$$E_a = |fl(x) - x|.$$

- **Error relativo** (para $x \neq 0$):

$$E_r = \frac{|fl(x) - x|}{|x|}.$$

En aritmética IEEE 754, utilizando redondeo al más cercano, se cumple que:

$$E_r = \frac{|fl(x) - x|}{|x|} \leq \frac{1}{2} \varepsilon_m,$$

donde ε_m es el épsilon de máquina visto en la Sección 3.5. Aquí vemos la relación entre este parámetro y la precisión de la máquina.

Demostración

Sea $x \neq 0$, se puede escribir en forma normalizada como

$$|x| = m \times 2^e, \quad \text{con } 1 \leq m < 2.$$

En un sistema de punto flotante con t bits en la parte fraccionaria del significante, los números representables en el intervalo $[2^e, 2^{e+1})$ están uniformemente espaciados con paso

$$\Delta = 2^{e-t}.$$

Al aplicar redondeo al más cercano, el error absoluto queda acotado por la mitad de ese paso:

$$E_a = |fl(x) - x| \leq \frac{1}{2} \Delta = 2^{e-t-1}.$$

Dividiendo por $|x| = m \times 2^e$, se obtiene:

$$E_r = \frac{|fl(x) - x|}{|x|} \leq \frac{2^{e-t-1}}{m \times 2^e} = \frac{1}{2 \times m} 2^{-t}.$$

Como $1 \leq m < 2$, resulta:

$$E_r = \frac{|fl(x) - x|}{|x|} \leq \frac{1}{2} 2^{-t}.$$

Recordando que $\varepsilon_m = 2^{-t}$, se concluye que:

$$E_r = \frac{|fl(x) - x|}{|x|} \leq \frac{1}{2} \varepsilon_m.$$

Ejemplo

Empleando el método de conversión, se puede determinar que $(3.45)_{10}$ no tiene representación exacta. Su valor aproximado en el formato de simple precisión es $(0|10000000|10111001100110011001101)_2$, que corresponde al valor $(3.4500000476837158203125)_{10}$, y el error cometido por representación es aproximadamente 4.77×10^{-8} . Por lo tanto, el error relativo es aproximadamente 1.38×10^{-8} , el cual está acotado por $\frac{1}{2} \epsilon_m \approx 5.96 \times 10^{-8}$.

6.5. Precauciones y consecuencias al operar con números en punto flotante

“La precisión es finita; los errores, acumulativos.”

La alineación o ajuste de exponentes se realiza desplazando el significante del número de menor exponente hacia la derecha (incrementando su exponente). Este proceso puede implicar la pérdida de bits menos significativos, los cuales se descartan de acuerdo con la regla de redondeo del estándar IEEE 754. Por este motivo, conviene alinear siempre respecto del número de mayor magnitud, ya que los bits perdidos tienen menor peso relativo.

Ejemplo

Queremos sumar $S = N_1 + N_2$ con $N_1 = (1230.015625)_{10}$ y $N_2 = (4.56)_{10}$ en formato IEEE 754 de simple precisión. Convirtiendo los números N_1 y N_2 se obtiene:

$$N_1 = (0|10001001|001100111100000010000000)_2$$

$$N_2 = (0|10000001|00100011110101110000101)_2$$

El exponente del primer número es $e_1 = 10$, mientras que el del segundo es $e_2 = 2$. Por lo tanto, es necesario alinear los exponentes desplazando el significante de N_2 ocho posiciones hacia la derecha. Como consecuencia, los bits menos significativos se descartan (con redondeo), introduciendo un error de aproximadamente $2^2 \times (2^{-16} + 2^{-21} + 2^{-23}) = 0.6342 \times 10^{-4}$.

Por el contrario, si se hubiera alineado N_1 respecto de N_2 , el error habría sido significativamente mayor: $2^{10} \times 2^{-16} = 0.1563 \times 10^{-1}$.

A partir de lo anterior se desprenden las siguientes consideraciones prácticas:

1. Se obtiene mayor precisión al operar con números de magnitud similar.
2. Al restar números muy próximos puede producirse **cancelación catastrófica**, donde gran parte de las cifras significativas se pierde y el resultado queda dominado por el error de redondeo.
3. El orden de evaluación afecta la precisión: en general, $A \oplus (B \oplus C) \neq (A \oplus B) \oplus C$.
4. La propiedad distributiva no se preserva en punto flotante: $x \otimes (y \oplus z) \neq x \otimes y \oplus x \otimes z$.
5. En cadenas de cálculos, conviene priorizar productos y cocientes antes que sumas o restas, y agrupar operaciones de manera que se eviten combinaciones de magnitudes muy dispares.
6. No debe asumirse igualdad exacta entre números en punto flotante. Las comparaciones deben realizarse utilizando tolerancias adecuadas, por ejemplo:

$$|x - y| \leq \text{tol} \quad \text{o} \quad \frac{|x - y|}{|y|} \leq \text{tol},$$

considerando casos límite como $x = y = 0$ o $y = 0$.

Estas consideraciones están directamente relacionadas con el error de redondeo y el épsilon de máquina (ϵ_m), y motivan el diseño de algoritmos numéricamente estables.

Ejemplo

El siguiente ejemplo ilustra que la comparación exacta entre números en punto flotante no es fiable.

```
#include <stdio.h>
#include <math.h>

int main() {
    double a = 0.1;
    double b = 0.2;
    double c = 0.3;

    printf("a + b = %.17f\n", a + b);
    printf("c = %.17f\n", c);
```

```
if (a + b == c)
printf("Son iguales\n");
else
printf("No son iguales\n");

if (fabs((a + b) - c) < 1e-15)
printf("Son aproximadamente iguales\n");

return 0;
}
```

Los valores 0.1, 0.2 y 0.3 no tienen representación exacta en base 2. En IEEE 754 se almacenan como aproximaciones binarias. Al realizar la suma, los errores de redondeo se propagan, produciendo un resultado distinto de la representación de 0.3.

A. Apéndices

A.1. Formatos y características de punto flotante (IEEE 754)

En la Tabla 2 se muestra un resumen de las características de los números en punto flotante de la norma IEEE 754 en precisión simple, doble y cuádruple.

A.2. Cálculo del ϵ de máquina

El ϵ de máquina depende del formato de representación en punto flotante utilizado, el cual está determinado por la arquitectura del hardware y su implementación (por ejemplo, IEEE 754 en simple, doble o mayor precisión), ya que está directamente relacionado con la cantidad de bits del significante.

El siguiente código permite calcular el ϵ de máquina para diferentes formatos (compilar con la opción `-lquadmath`):

Código

```
#include <stdio.h>
#include <float.h>
#include <quadmath.h>

int main(){
    float x = 1.0;
    double y = 1.0;
    long double z = 1.0;
    _Float128 w = 1.0;
    int n = 0;
    char buf[128];

    while ((float)1.0 + (x * (float)0.5) > (float)1.0){
        ++n;
        x *= 0.5;
    }
    printf("Float: Épsilon de máquina: 2(- %d)=%g\n", n, x);

    n = 0;
    while (1.0 + (y * 0.5) > 1.0){
        ++n;
        y *= 0.5;
    }
```

Tabla 2: Características de los números en punto flotante (Standard IEEE 754).

	Precisión Simple	Precisión Doble	Precisión Cuádruple
Bits de signo	1	1	1
Bits de exponente	8	11	15
Bits de fracción	23	52	112
Bits totales	32	64	128
Sesgo del exponente	+127	+1023	+16383
Rango del exponente	$[-126, 127]$	$[-1022, 1023]$	$[-16382, 16383]$
Valor más chico (normalizado)	2^{-126}	2^{-1022}	2^{-16382}
Valor más grande (normalizado)	$\approx 2^{128}$	$\approx 2^{1024}$	$\approx 2^{16382}$
Épsilon de máquina	$2^{-23} \approx 1.19 \times 10^{-7}$	$2^{-52} \approx 2.22 \times 10^{-16}$	$2^{-112} \approx 1.92 \times 10^{-34}$
Rango decimal	$[\approx 10^{-38}, \approx 10^{38}]$	$[\approx 10^{-308}, \approx 10^{308}]$	$[\approx 10^{-4932}, \approx 10^{4932}]$
Valor más chico (desnormalizado)	$2^{-126} \times 2^{-23} \approx 10^{-45}$	$2^{-1022} \times 2^{-52} \approx 10^{-324}$	$2^{-16382} \times 2^{-112} \approx 10^{-4966}$
Tipo de dato en C	float	double	_Float128

```

    }
    printf("Double: Épsilon de máquina: 2(-%d)=%g\n", n, y);

    n = 0;
    while (1.0 + (z * 0.5) > 1.0){
        ++n;
        z *= 0.5;
    }
    printf("Long double: Épsilon de máquina: 2(-%d)=%Lg\n",
        n, z);

    n = 0;
    while (1.0 + (w * 0.5) > 1.0){
        ++n;
        w *= 0.5;
    }

    quadmath_snprintf(buf, sizeof buf, "%.4Qg", w);
    printf("_Float128: Épsilon de máquina: 2(-%d)=%s\n", n, buf);

    return 0;
}

```

Observación

El tipo `long double` depende de la implementación del compilador y la arquitectura, por lo que su precisión puede variar (por ejemplo, 80 bits en arquitecturas x86 o 64 bits en otros sistemas). En cambio, el tipo `_Float128` representa un formato bien definido de precisión cuádruple (IEEE 754 binary128), con una cantidad fija de bits para el significante. Por lo tanto, `_Float128` permite trabajar con una precisión conocida y portable, mientras que `long double` no garantiza una precisión uniforme entre distintas plataformas.

A.3. Ejemplo en C: parámetros en punto flotante

El siguiente programa en C muestra algunos parámetros fundamentales de la aritmética en punto flotante definidos en `<float.h>`, tanto para simple como para doble precisión.

Código

```
#include <stdio.h>
#include <float.h>

int main() {
    printf("=== SIMPLE PRECISION (float) ===\n");
    printf("Epsilon (FLT_EPSILON)      = %.10e\n",
           FLT_EPSILON);
    printf("Min positivo normalizado    = %.10e\n", FLT_MIN);
    printf("Min positivo (incluyendo subnormales) = %.10e\n",
           FLT_TRUE_MIN);
    printf("Max valor representable    = %.10e\n", FLT_MAX);
    printf("Digitos de precision        = %d bits\n",
           FLT_MANT_DIG);

    printf("\n=== DOBLE PRECISION (double) ===\n");
    printf("Epsilon (DBL_EPSILON)      = %.20e\n",
           DBL_EPSILON);
    printf("Min positivo normalizado    = %.20e\n", DBL_MIN);
    printf("Min positivo (incluyendo subnormales) = %.10e\n",
           DBL_TRUE_MIN);
    printf("Max valor representable    = %.20e\n", DBL_MAX);
    printf("Digitos de precision        = %d bits\n",
           DBL_MANT_DIG);
    return 0;
}
```

```
===== SIMPLE PRECISION (float) =====
Epsilon (FLT_EPSILON)      = 1.1920928955e-07
Min positivo normalizado    = 1.1754943508e-38
Min positivo (incluyendo subnormales) = 1.4012984643e-45
Max valor representable    = 3.4028234664e+38
Digitos de precision        = 24 bits
```

```
===== DOBLE PRECISION (double) =====
Epsilon (DBL_EPSILON)      = 2.22044604925031308085e-16
Min positivo normalizado    = 2.22507385850720138309e-308
Min positivo (incluyendo subnormales) = 4.9406564584e-324
Max valor representable    = 1.79769313486231570815e+308
Digitos de precision        = 53 bits
```

A.4. Manipulación de la representación binaria de números en punto flotante

Como hemos visto en el apunte anterior, los operadores de bits pueden aplicarse a variables de tipo entero, tanto con signo como sin signo (`char`, `short`, `int`, `long`, etc.), y no pueden aplicarse a variables de tipo flotante (`float`, `double`, etc.). No obstante, es posible manipular la representación binaria de un número flotante reinterpretando su contenido como un entero del mismo tamaño (por ejemplo, mediante punteros, `union` o `memcpy`), evitando así una conversión de valor.

- **Opción usando con punteros:**

```
float f = 3.14f;
unsigned int bits = *(unsigned int*)&f;
```

Ahora `bits` contiene los mismos bits que `f` (formato IEEE 754), y se pueden aplicar operadores de bits, pero puede producir comportamiento indefinido en compiladores optimizados.

- **Opción usando `memcpy`:**

```
#include <string.h>

float f = 3.14f;
unsigned int bits;

memcpy(&bits, &f, sizeof(bits));
```

Esta opción garantiza un comportamiento definido.

- **Opción clásica usando `union`:**

```
union {
    float f;
    unsigned int u;
} x;

x.f = 3.14f;
unsigned int bits = x.u;
```

Esto es muy usado en la práctica, pero su portabilidad depende de detalles del compilador.

Ejemplo

```
#include <stdio.h>
#include <stdint.h>

int main() {
    float f = -12.5f;
    // Forzamos al compilador a leer los bits del float
    // como un entero
    uint32_t bits = *(uint32_t*)&f;

    uint32_t signo = (bits >> 31) & 1;
    uint32_t exponente = (bits >> 23) & 0xFF;
    uint32_t mantisa = bits & 0x7FFFFFFF;

    printf("Representación hexadecimal completa: 0x%08X\\n", bits);
    return 0;
}
```

Ejemplo

```
#include <stdio.h>
#include <stdint.h>

// Definimos la estructura exacta de un float de 32 bits
// según IEEE 754
typedef union {
    float f;
    struct {
        uint32_t mantisa : 23;
        uint32_t exponente : 8;
        uint32_t signo : 1;
    } bits;
} IEEE754_Float;

int main() {
    IEEE754_Float numero;
    numero.f = -12.5f; // Número de prueba
}
```

```

    printf("Signo: %u\n", numero.bits.signo); // 1 (
        Negativo)
    printf("Exponente (con sesgo): %u\n", numero.bits.
        exponente); // 130
    printf("Mantisa (en hex): 0x%X\n", numero.bits.
        mantisa); // 0x480000

    return 0;
}

```

Referencias

- [1] Mano, M.M., *Computer System Architecture*, Prentice-Hall, 1993.
- [2] Hyde, R., *The Art of Assembly Language*, No Starch Pr, 2003.
- [3] Goldberg, D, *What Every Computer Scientist Should Know About Floating-Point Arithmetic*, ACM Computing Surveys (CSUR), **(23)**, 5-48, 1991.
- [4] IEEE Computer Society, *IEEE Standard for Floating-Point Arithmetic*, 2008.
- [5] Kahan, W., *Lecture Notes on the Status of IEEE Standard 754 for Binary Floating-Point Arithmetic*, 1996.